REPORT 2: UML CLASS DIAGRAM REPORT **Project Title:** Online Library Management System (OLMS) **Institution:** Kigali Independent University (ULK)[cite: 1]

1. Complete UML Class Diagram Structure

The system architecture is modeled using Object-Oriented principles, highlighting generalization, association, and composition relationships. ### 2. Classes, Attributes, and Methods Summary

| Class Name | Attributes (Data Members) | Methods (Operations) | | :--- | :--- | :--- | | **`User`** *(Abstract)* | `- userId: String`
`- name: String`
`- email: String`
`- phone: String` | `+ login(): Boolean`
`+ logout(): void` | | **`Member`** *(inherits User)* | `- membershipId: String`
`- maxBooksAllowed: int` | `+ searchBook(title: String): List`
`+ reserveBook(bookId: String): void` | | **`Librarian`** *(inherits User)*| `- employeeId: String`
`- shift: String` | `+ addBook(book: Book): void`
`- removeBook(bookId: String): void`
`- issueBook(memberId, bookId): BorrowRecord` | | **`Book`** | `- bookId: String`
`- title: String`
`- author: String`
`- ISBN: String`
`- isAvailable: boolean` | `+ markAsBorrowed(): void`
`- markAsReturned(): void` | | **`BorrowRecord`** | `- recordId: String`
`- issueDate: Date`
`- dueDate: Date`
`- returnDate: Date` | `+ calculateOverdueDays(): int`
`- updateReturnDate(date: Date): void` | | **`Fine`** | `- fineId: String`
`- amount: double`
`- isPaid: boolean` | `+ generateFine(): double`
`+ processPayment(): void` | ### 3. Relationships and Multiplicities Explained 1. **Generalization (Inheritance):** * `Member` and `Librarian` inherit from the abstract class `User`. This allows code reusability for shared properties like names, contact details, and login functionality. 2. **Member to BorrowRecord (Association):** * **Multiplicity:** `1` Member to `0..*` BorrowRecord. * **Explanation:** A registered library member can place zero or multiple borrowing transactions over time. 3. **Librarian to BorrowRecord (Association):** * **Multiplicity:** `1` Librarian to `0..*` BorrowRecord. * **Explanation:** A librarian manages and processes multiple borrowing and returning transactions. 4. **Book to BorrowRecord (Association):** * **Multiplicity:** `1` Book to `0..*` BorrowRecord. * **Explanation:** A specific book title/item can be referenced across multiple past or present borrowing records. 5. **BorrowRecord to Fine (Composition):** * **Multiplicity:** `1` BorrowRecord to `0..1` Fine. * **Explanation:** A borrowing record can incur at most one fine if the book is returned past the due date. The fine's lifecycle is tightly bound to the loan record.